Changing behavior by file type: http://stackoverflow.com/questions/158968/changing-vim-indentation-behavior-by-file-type

Using buffers correctly: https://stackoverflow.com/questions/102384/using-vims-tabs-like-buffers
Splitting up into multiple files: Splitting up vimrc into multiple files: https://www.reddit.com/r/vim/comments/31v6uj/simple_elegant_way_of_splitting_up_vimrc/

[CLI] Opening in read-only
vim -M *file* Open in read-only

[CLI] Edit remote files (scp & netrw)
https://gist.github.com/RRethy/ad8a9a3b1112a48226ec3336fa981224

```
"" Open a file
vim scp://my.server/myfile.txt             " ~/myfile.txt
vim scp://myserver//home/yui/myfile.txt    " Absolute path
vim scp://yui@myserver:1234/myfile.txt     " user and port

vim scp://myserver/.config          // open directory
vim scp://myserver/                 // open $HOME
```

[buffer] Opening current buffer in a new tab/window
```
:tabe %
```

[buffer] Create temporary buffer
Create a new buffer with buftype=nofile

```
:badd mybuf                 " create buffer
:buffer mybuf               " switch to buffer
:setlocal buftype=nofile
```

[buffer] Save all existing buffers
Use bufdo

```
:bufdo write
```

[buffer] Run command in existing buffers
Use bufdo

```
" e.g. substitute
:bufdo %s/aaa/bbb
```

[buffer] Delete all buffers except current window
Use :bd with range

```
:1,.-bd        Delete all before current (assuming 1=first buffer)
:.+,$bd        Delete all after current
```

[buffer] Get buffer numbers: current, first, last
%=current
$=last
No simple expression for first buffer

```
' Current buffer number
:let l:current=bufnr('%')        Current buffer

' Last buffer number
:let l:last=bufnr('$')           Last buffer

' First buffer number
:let l:first=bufnr('^')          NO. Checks if buffer with name '^'
:let l:first=1
for i in range(1,bufnr('$'))
 if bufexists(i)
  let l:first=i
  break
 endif
endfor
```

Edit/reload vimrc

```
:edit $MYVIMRC
:so $MYVIMRC
```

Show windows characters

```
:set fileformats=unix
```

Implement logging

```
let g:yt_logdir='/tmp'
let g:yt_logpath=yt_logdir . '/yt.log'

function! YT_writeLog(message)
 ' strftime=timestamp
  call writefile([strftime('%c') . ' - ' . a:message],
g:yt_logpath, 'a')
endfunction

" note: initialize log file
" writefile() fails if the file does not exist
function! YT_initLog()
  call system('mkdir -p ' . g:yt_logdir . ' && touch ' .
g:yt_logpath)
endfunction

call YT_initLog()

" Log
" In /tmp/yt.log
call YT_writeLog('hello')        " entry=Sun  8 Aug 12:49:38 2021 -
hello
```

[shell] Executing a shell command
1) Ctrl-z to move vim to background.
2) Execute command
3) fg to return to vim.

[shell] read and write environment variables

```
# Read
echo $PATH

# Write
let $PATH=$PATH . "$HOME/bin:"
```

[shell] Put shell command output into a variable
Use system()

```
" Store in variable x
let x=system("ls")
```

[shell] Put shell command output into a register
Ref: https://stackoverflow.com/questions/1694392/vim-store-output-of-external-command-into-a-register
Use system() and register variable @<reg>

```
" Store in register 'c'
let @c=system("ls")
```

[shell] Pass content of file to a shell command
Use :write !<shell command>

```
:w !cat
:w !cat > /tmp/file          " Can redirect
```

Set transparent background
e.g. when to prefer using terminal background
See https://stackoverflow.com/questions/37712730/set-vim-background-transparent

" Keep background transparent
" To avoid conflict with background of vim and the terminal
hi Normal guibg=NONE ctermbg=NONE

Set filetype on file read

```
autocmd BufRead,BufNewFile *.html,*.xhtml set filetype=html
autocmd BufRead,BufNewFile *.html,*.xhtml setfiletype html
```

Execute command for file types
Use autocmd

```
" BAD: doesn't work, since filetype isn't set yet.
if &filetype == "php"
     echo "Hello"
endif
```

```
" GOOD
autocmd FileType php echo "Hello"
autocmd FileType python echo "Hello"
```

Autocomplete css
Set omnifunc if not set.
Then hit C-X C-O

```
set omnifunc=csscomplete#CompleteCSS
```

==========

Misc

See previous errors e.g. plugin install errors, command errors
Use :message

```
# Show errors
:message
```

See compilation options e.g. python 2 or 3
Printed with vim version

```
# From CLI
vim --version

# From inside vim
:version
```

Get timestamp
Use the date command

```
" With !
!date +\%s           " Need to escape %

" With strftime
let l:timestamp = strftime('%')

" With system
let l:timestamp = system("date +%s")
```